

תרגיל 4 – הערות כלליות

ההערות במסמך נוגעות לטעויות חוזרות שנצפו במהלך הבדיקה, ומחולקות לפי סדר השאלות בתרגיל.

שאלה 2

- עץ קריאות רקורסיביות – על מנת לבדוק את עץ הקריאות ניתן פשוט להוסיף הדפסה פשוטה שמתבצעת בכניסה לפונקציה (חשוב לזכור: להסיר את ההדפסה לפני ההגשה!). הניסיון לנתח על דף בכוחות עצמנו את עץ הקריאות לפי הבנתנו את הפונקציה הוא חשוב, אבל כדאי לגבות אותו בבדיקה כנ"ל. בנוסף, סטודנטים רבים שכחו לציין בעץ הקריאות בדיקות עבור $size=0$
- קריאות מיותרות – נתבונן בשתי פיסות פסאודו-קוד דומות. על פניו ברור כי התוצאה הסופית תהיה זהה, אך חשוב להבחין כי במימוש הראשון מתבצעות קריאות רקורסיביות מיותרות – כל השוואה גוררת קריאה רקורסיבית כפולה, מה שגורר בסופו של דבר גידול אקספוננציאלי במספר הקריאות לפונקציה -
 - for i in range(n):
 - if val+profit(params) > max_profit:
 - max_profit = val+profit(params)
 - for i in range(n):
 - max_profit = max(max_profit, val+profit(params))
- שימוש חוזר בפונקציות – במספר מקרים סטודנטים מימשו את profit_mem באמצעות קריאה ל-profit. הקונספט של שימוש חוזר בפונקציות קיימות על פני מימוש כפול הוא נכון וחשוב, אבל יש להבחין כי במקרה הנוכחי קריאה שכזו תוביל להפעלה של הפונקציה profit2 ללא ממואיזציה (שכן לאחר הקריאה הראשונית ל-profit נשאר במימוש המקורי). באופן זה, קריאה מ-profit_mem לפונקציה profit2 לא תועיל, מאחר והמילון ב-profit2 מאופס תמיד
- במספר הגשות צוין כי עץ הקריאות של profit, profit2 – תופעה שכזו צריכה להדליק נורת אזהרה, שכן לפיה – אין משמעות לשימוש בממואיזציה. במצב זה ישנן שתי אפשרויות – או שהניתוח של העץ איננו נכון, או שהשימוש בממואיזציה לא מומש כיאות (ראו סעיף קודם)

שאלה 4

- העברה גלויה של הסוד – חשוב להבין כי מטרת פרוטוקול DH היא לאפשר למספר משתתפים לחלוק "סוד" אשר אינו נתון למאזינים אשר מחזיקים מידע חלקי. במספר הגשות בוצע תהליך תקין של בניית הסוד על ידי כל משתתף, ולבסוף צוין כי כל משתמש שולח את הסוד למשתמש אחר. ניתן לשים לב כי מאחר וכל משתמש חישב את הסוד אין שום צורך להעבירו למשתמש אחר, אך חשוב מכך – פעולה זו מבטלת כל יתרון של שימוש בפרוטוקול שכן כעת מאזין על התווך יחזיק את הסוד!
- יעילות של פעולות אריתמטיות – מעבר להנחות לגבי בטיחות, הפרוטוקול מסתמך על יעילות ביצוע פעולת iterated squaring.
- פתרון מילולי/דיאגרמה – הסבר מילולי לפרוטוקול הוא לגיטימי, כל עוד הפרוטוקול מתואר באופן ברור ותמציתי. באם הפתרון לא ברור לחלוטין – אפשר ורצוי להוסיף דיאגרמה (בדומה לאחת שהוצגה בתרגיל)
- חשוב לציין כי משתמש המחזיק במפתח a אשר ברשותו סוד חלקי g^k מסוגל לבצע ביעילות את הפעולה $(g^k)^a = g^{ka} \mod p$. מאידך אם ברשות המשתמש סודות חלקיים g^l, g^k , אנו מניחים כי אין ביכולתו להרכיב מהם ביעילות את הסוד המלא $g^{kl} \mod p$

שאלה 5

- מספר סטודנטים הציגו טענה הבנויה באופן הבא: פעולת החישוב מבצעת k איטרציות, עלות איטרציה הינה ℓ , ולכן העלות הכוללת של האלגוריתם היא $O(k + \ell)$. בחישוב שכזה נסיק כי העלות הכוללת היא $O(k \cdot \ell)$
- במספר מקרים נטען כי סיבוכיות חיבור בין שני מספרים שהינה לינארית בגודל הקלט זהה לסיבוכיות כפל ביניהם. בשאלה הנוכחית סיבוכיות הכפל בין שני מספרים בעלי m, n ביטים מתבצעת בזמן $O(m \cdot n)$
- סטודנטים רבים טענו כי סיבוכיות האלגוריתם הינה: $O(2^m \cdot n^2) = O(2^m \cdot n^2)$. חשוב לשים לב כי: $(2^m)^2 = 2^{2m} = 4^m \neq O(2^m)$
- טעות חוזרת נוספת הייתה התייחסות לגודל היצוג של המשתנה הזמני a כקבועה על n לאורך כל החישוב. מאחר וכל פעולת העלאה בחזקה עלולה להכפיל את גודל היצוג של a , נקבל כי לאחר m איטרציות גודלו חסום ב- $2^m n$.

שאלה 6

- סטודנטים רבים השתמשו בכלים אוטומטיים לחישוב פונקציה הנותנת קירוב למדדים אותם התבקשתם לנתח, דוגמת רגרסיה באקסל. חשוב להבחין כי התאמה אשר ניתנת על ידי כלי שכזה לאו דווקא נותנת ניתוח "הגיוני" של המדדים אותם נתבקשתם לנתח. לדוגמא, קירוב לפונקציה אשר מתארת פונקציות הסתברות אך מאפשרת קבלת ערכים שליליים ו/או גדולים מ-1. מטרת הניסוי הייתה לקבל תחושה ואינטואיציה לגבי אופי המדדים כתלות במרחב בו מתבצע ההילוך ואורכו

שאלה 1 – משוב קוד פונקציה "acc"

שם ה-Cluster הרלוונטי לפתרונות וטכניקות שמוצגות במסמך מופיע בסוגריים עם Highlight. במשוב האישי לתרגיל ניתן למצוא את שם ה-Cluster המתאים לתשובתכם. מומלץ לעבור על כל המסמך ולא רק על החלקים הנוגעים ל-Cluster שלכם.

בתרגיל הנתון נתבקשתם להפעיל פונקציה נתונה באופן רקורסיבי על גבי מערך של ארגומנטים בהנתן ערך התחלתי. סדר הצבת הארגומנטים ואופן הקריאה הרקורסיבית מושפע ישירות מאופיה של הפונקציה הנתונה – בהנתן פונקציה לא-קומוטטיבית, סדר הצבת הערכים בפונקציה הוא קריטי ($f(x, y) \neq f(y, x)$), בהנתן פונקציה לא אסוציאטיבית, סדר ההצבה של הארגומנטים במערך הוא קריטי ($f(f(x_1, x_2), x_3) \neq f(f(x_3, x_2), x_1)$), וכו'. מאחר ובהוראות התרגיל לא נדרשתם במפורש להתמודד עם פונקציות בעלות תכונות אלו, לא ירדו נק' על מימוש שאינו תומך בפונקציות אלו. עם זאת, ניתוח הקוד נותן לנו הזדמנות מעולה לאמוד על ההבדלים הדקים בין מימוש אחד ומשנהו והאופן בו הם משפיעים על התוצר הסופי.

אסוציאטיביות

כאמור, בפונקציה שאינה אסוציאטיבית סדר הפעלת האיברים במערך הינו קריטי, ולכן נבחין כי שני המימושים הבאים אינם שקולים. בעוד המימוש הראשון (Non Associative) סוקר את איברי המערך מסופו לתחילתו:

```
def acc(f, v, lst):
    if len(lst)==0:
        return v
    else:
        return acc(f, f(lst[-1],v) ,lst[:len(lst)-1])
```

המימוש השני סוקר את איברי המערך מתחילתו לסופו (Commutative+Associative):

```
def acc(f, v, lst):
    if len(lst) == 0:
        return v
    else:
```

```
return acc(f, f(v, lst[0]), lst[1:])
```

קומוטטיביות

כאמור, בפונקציה שאינה קומוטטיבית סדר הצבת הארגומנטים בפונקציה הינו קריטי, ולכן הקריאות הרקורסיביות הבאות יחזירו תוצאות שונות. ראשית, קריאה קומוטטיבית:

```
def acc(f, v, lst):
    if len(lst) == 0:
        return v
    else:
        return acc(f, f(v, lst[0]), lst[1:])
```

ושנית, קריאה שאינה קומוטטיבית (Non Commutative), כאשר השינוי הדק מודגש לנוחות הקריאה:

```
def acc(f, v, lst):
    if len(lst) == 0:
        return v
    else:
        return acc(f, f(lst[0], v), lst[1:])
```

רקורסיה על f

במקרים רבים הוצע פתרון בעל אופי דומה לפתרון הבא (Recursion on f):

```
def acc(f, v, lst):
    if len(lst) == 0:
        return v
    else:
        return f(v, acc(f, lst[0], lst[1:]))
```

על מנת להבין את ההשפעה של מימוש זה, ניתן למשל לבחון את ההפעלה שלו על הפונקציה $f(x, y) = x^y$. נתבונן למשל בקריאה ל-`acc(lambda x,y:x**y,2,[3,4,5])`. הקריאה הרקורסיבית הראשונה תחזיר לנו את הערך:

$$f(2, acc(f, 3, [4, 5])) = 2^{acc(f, 3, [4, 5])}$$

וכן הלאה. בעת החזרה מהרקורסיה, נבחין כי התוצאה הסופית שנקבל הינה:

$$2^{(3^{(4^5)})}$$

בעוד קריאה אסוציאטיבית היתה מחזירה את התוצאה:

$$((2^3)^4)^5$$

נסיק, אם כן, כי גם במקרה זה המימוש השפיעה על אסוציאטיביות הקוד. בנוסף, נבחין כי אם סדר הארגומנטים בקריאה הרקורסיבית ל-f היה שונה, אזי היינו מקבלים גם השפעה על קומוטטיביות הפעולה, לדוגמא, עבור קריאה רקורסיבית ל:

```
f(acc(f, lst[0], lst[1:]), v)
```

מקרי קצה מיותרים

נבחן שני מימושים שקולים לפונקציה. הראשון (Commutative+Associative):

```
def acc(f, v, lst):
    if len(lst) == 0:
        return v
    else:
        return acc(f, f(v, lst[0]), lst[1:])
```

לעומת:

```
def acc(f, v, lst):
    if not lst:
        return v
```

```

if len(lst) == 1:
    return f(lst[0], v)
if len(lst) == 2:
    return f(f(v, lst[0]), lst[1])
else:
    return acc(f, f(v, lst[0]), lst[1:])

```

ראשית נציין כי שני המימושים תקינים לחלוטין. עם זאת, נרצה לעמוד על מספר הבדלים:

המימוש הראשון אלגנטי וקריא. בנוסף, הוא מציג גם הבנה מלאה של הקונספט הרקורסיבי. מקרה הבסיס הנאיבי הינו מצב בו רשימת הארגומנטים ריקה ולכן נחזיר את הערך ההתחלתי שקיבלנו. במימוש השני אנו מבדילים בין רשימה ריקה ובין "רשימה קצרה", אך הבדל זה יתיר

בנוסף, אע"פ שהמימוש הנ"ל תקין, נבחין כי הוא מעלה את הסבירות לטעות קידוד – באם הפונקציה שלנו היתה לא-קומוטטיבית $a * b \neq b * a$, כפי שתיארנו קודם, אזי השורות הבאות אינן שקולות:

```

return f(f(v, lst[0]), lst[1])  $\neq$  return f(f(lst[1], lst[0]), v)

```

במצב שכזה, עולה הסבירות לטעות בסדר ההפעלה בקריאה הנ"ל, מה שיכול לפגוע בנכונות הקוד במקרים מסויימים.